Problem 7. [32 points] **Zero Sum**

(a) [16 points] Given $2k$ arrays, where each array contains exactly n integers, describe an $O(n^k)$-time algorithm to determine whether there exists $2k$ integers, exactly one from each array, that sum to zero. State whether your running time is worst-case, expected, and/or amortized. Significant partial credit will be given for $O(kn^k)$-time algorithms. (Hint: First try solving for small k.)

Solution: Let $S(i, j)$ be the set of sums of all $O(n^{j-i+1})$ subsets that could be formed by taking one integer from each of the arrays from array i to array j. If we could compute $S(1, k)$ and $S(k + 1, 2k)$ in $O(n^k)$ time, we could insert each sum from $S(1, k)$ into a hash table in expected $O(1)$ time, and then look up the negative of each sum in $S(k+1, 2k)$ to certify whether $S(1, 2k)$ contains zero in expected $O(n^k)$ time.

Naively, we could compute $S(1, k)$ in $O(kn^k)$ time by computing each subset sum of k integers directly. Alternatively, notice that we can compute the $O(n^i)$ sums in $S(1, i)$ from $S(1, i - 1)$ by adding a single integer from array i to one of the $O(n^{i-1})$ sums from $S(i-1)$ in $O(1)$ time, and $O(n^i)$ time in total. Then computing $S(1, k)$ can be done by computing $S(i)$ in order for $i \in \{1, \ldots, k\}$ in time $O(\sum_{i=1}^{k} n^i) = O(n^k)$ as desired, with $S(k + 1, 2k)$ computed in a similar fashion.

Common Mistakes:

- Not computing sums efficiently
- (or not recognizing that naive sums take $O(k)$ time)

Problem 6. [16 points] **Mostly-Blue**

- A **blue-labeled** graph is a directed graph where every edge is labeled either blue or not-blue.
- A cycle in a blue-labeled graph is **mostly-blue** if more than half of its edges are blue.
- A vertex is **high-degree** if the sum of its in-degree and out-degree is strictly more than two.

Given a blue-labeled simple graph G containing n vertices and at most $O(n)$ edges, where exactly $k < n$ vertices are high-degree, describe an $O(n + k^3)$-time algorithm to determine whether G contains a mostly-blue cycle.

Solution: Observe that every connected component in G either (a) contains a high-degree vertex, (b) is a directed cycle, or (c) does not contain a directed cycle. Run a full depth-first search of G in $O(n)$ time to identify the connected components of the graph and categorize each one as above. (c) components may be safely ignored. A (b) component is a mostly-blue cycle if more than half of its edges are blue, which can be checked in $O(n)$ time.

It remains to check for mostly-blue cycles in the (a) components. Without loss of generality, assume G contains only (a) components. Let the **blue-difference** of a directed path in a blue-labeled graph be the number of non-blue edges on the path minus the number of blue ones. Construct a new directed graph G' on the k high-degree vertices, with an edge from high-degree vertex u to high-degree vertex v of weight w if v is reachable from u along a path having only low-degree vertices between them, where w is the minimum blue-difference of any such path.

To identify these edges, split every high-degree vertex in G with degree d into d vertices, one attached to each adjacent edge. This new graph G'' is a set of disjoint chains. For each chain, run depth-first search from each endpoint to identify whether the chain is directed consistently from one endpoint to the other, and if so, compute its blue-difference directly in time proportional to the length of the chain. Doing this for each chain, we can construct the edges in G' in $O(n)$ time, using a hash table to retain the minimum blue-difference of all edges having the same start and end.

Now, G contains a mostly-blue cycle if and only if G' contains a negative-weight cycle, since a path in G' of weight w corresponds to a path in G having $-w$ more blue edges than non-blue edges. G' has at most k^2 edges, so we can run Bellman-Ford on G' to detect negative-weight cycles in $O(k^3)$ time, leading to an $O(n + k^3)$-time algorithm in total.

Common Mistakes:

- Not assigning a source vertex when running a single-source shortest paths algorithm
- Running Bellman-Ford or Johnson's on a graph of size $\Theta(n)$
- Trying to enumerate all cycles (there may be an exponential number of cycles)
- Assuming DAG or trying to only detect unweighted cycles

Problem 3. [14 points] **Pams vs. Ratriots**

Bom Trady is a football fan who will be attending the Big Game™, where her home team, the Ratriots, will be playing rival team, the Pams. The game will be held in a neutral city, so Bom will fly there and walk to the stadium from the airport.

- Upon arriving, Bom discovers some streets are **occupied**. A street occupied by fans from team A may only be traversed while wearing a team A jersey, and possessing no other jersey.
- Bom has a map depicting the m two-way streets and n intersections in the city, where each street directly connects two intersections, and each intersection connects at most four streets. The airport and stadium intersections are both marked on the map.
- Each street is marked with its positive integer length and its occupied status: either Ratriots, Pams, or unoccupied. Some intersections on the map are marked as having shops, which sell jerseys from both teams.

Assume Bom begins wearing a Ratriots jersey, always has enough money to purchase jerseys, and may discard a jersey at any time. Given her map, describe an $O(n \log n)$-time algorithm to return a shortest occupation-respecting route from the airport to the stadium (or return no such route exists).

Solution: Construct a graph G with a vertex for each intersection in the map marked whether it contains a shop, and for each street between intersections a and b of length ℓ, add an undirected edge $\{a, b\}$ of weight ℓ, marked with the street's occupation status. G has n vertices and $m = O(n)$ edges (since each vertex connects to at most four edges), so can be constructed in $O(n)$ time.

Now construct two graphs G_r and G_p, where G_r is G with all Pams-occupied edges removed, and G_p is G with all Ratriots-occupied edges removed. Then for every intersection v that contains a shop, connect v_p from G_p to v_r from G_r via a weight-zero undirected edge to construct graph G' (which also takes $O(n)$ time to construct). Let s and t correspond to the airport and stadium intersections respectively. Then any occupation-respecting route from the airport to the stadium corresponds to a path from s_r to either t_r or t_p in G'. G' has only positive edge weights, so run Dijkstra's algorithm from s_r and return the shortest route to either t_r or t_p, which runs in $O(n \log n)$ time. If the minimum weight path has weight ∞, return that no path exists.

Common Mistakes:

- Not adequately specifying starting and ending nodes for a search
- Not adequately specifying weights
- Not adequately specifying edges corresponding to unoccupied streets